Reusing Conjugate-Gradient Krylov Information for GP Predictive Variance

A short proposal for GPyTorch and `linear_operator`

Felix Reich

October 2026

Problem and existing approach. Consider the classical Gaussian process regression model. We assume that the observations $(x_1, y_1), \dots, (x_n, y_n)$ satisfy

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \qquad i = 1, \dots, n,$$

where the noise variables are independent and $f \sim \mathcal{GP}(0, k)$ with a kernel k. We assume a zero prior mean only to simplify notation. For any two collections of input locations $Y = (u_i)_{i=1}^p$ and $Z = (v_\ell)_{\ell=1}^q$, write $K_{YZ} := k(Y, Z) = (k(u_i, v_\ell))_{i,\ell}$. Let $X = (x_1, \dots, x_n)$ denote the training inputs and let $y = (y_1, \dots, y_n)^\top$ denote the observation vector. Writing $\widehat{K} = K_{XX} + \sigma^2 I$, the posterior mean and covariance of the latent function at test inputs X_* are

$$\mu_* = K_{X_*X}\widehat{K}^{-1}y, \qquad \Sigma_* = K_{X_*X_*} - K_{X_*X}\widehat{K}^{-1}K_{XX_*}. \tag{1}$$

For large n, GPyTorch can use conjugate gradients (CG) to solve $\widehat{K}w = y$ for the posterior mean. With fast predictive variances enabled, LOVE [1] separately constructs an orthonormal Lanczos basis Q_J and a projected matrix $T_J = Q_J^\top \widehat{K} Q_J$. It then uses the approximation $\widehat{K}^{-1} \approx Q_J T_J^{-1} Q_J^\top =: C_J^{\mathrm{LOVE}}$ for the covariance correction. Thus the mean solve and variance approximation both use Krylov information, but the latter constructs its basis afresh.

From CG directions to a Lanczos inverse approximation. A covariance approximation based on CG directions was introduced in [2] and further studied in [3]. Let $d_1, \dots, d_j$ be the j linearly independent directions from an unpreconditioned CG solve. The approximation based on the algorithm in [2] is

$$\widehat{K}^{-1} \approx \sum_{i=1}^{j} \frac{d_i d_i^\top}{d_i^\top \widehat{K} d_i} =: C_J^{\mathrm{CG}}. \tag{2}$$

If CG solves $\widehat{K}w = y$ from a zero initial guess and Lanczos starts from y, their projected-inverse approximations coincide after j iterations in exact arithmetic. That is, $C_J^{\mathrm{LOVE}} = C_J^{\mathrm{CG}}$, provided neither iteration breaks down.[1] This suggests reusing the directions from the mean solve instead of constructing a separate Lanczos basis for the variance. In an idealized setting where both computations take j iterations and matrix-vector products dominate the cost, reuse can reduce roughly $2j$ products to j. This suggests a speedup approaching a factor of two.

Recovering and extending the basis. During the CG mean solve, we store the search directions and their matrix products under $\widehat{K}$. Because finite precision can destroy $\widehat{K}$-conjugacy, we reorthogonalize the CG search directions in the $\widehat{K}$-inner product, analogously to Euclidean reorthogonalization in Lanczos. We perform a reduced QR factorization of the stored directions and then apply a numerical rank cutoff. Let j denote the number of directions retained after the rank cutoff, and write $D_j = [d_1, \dots, d_j] = \widetilde{Q}_j R_j$ and $W_j = \widehat{K} D_j$. Define $\widetilde{T}_j = \widetilde{Q}_j^\top \widehat{K} \widetilde{Q}_j$. The projected matrix is recovered from the stored products by solving

$$R_j^\top \widetilde{T}_j = W_j^\top \widetilde{Q}_j. \tag{3}$$

[1] See Theorem .1.

The retained QR basis spans the same subspace as the retained CG directions and, in exact arithmetic under the conditions above, a j-step Lanczos basis started from y. The recovered pair $(\widetilde{Q}_j, \widetilde{T}_j)$ can therefore be used for the same projected-inverse approximation. If a larger basis is useful, a Lanczos extension continues from the recovered subspace to a target size $J > j$. Writing $\widetilde{Q}_J$ and $\widetilde{T}_J$ for the resulting basis and projected matrix, an eigendecomposition of the stabilized matrix $\widetilde{T}_J = U\Lambda U^\top$ gives the inverse-root factor $B = Q_J U\Lambda^{-1/2}$ (just as for the LOVE implementation). The predictive covariance is then approximated by

$$\Sigma_* \approx K_{X_*X_*} - (K_{X_*X}B)(K_{X_*X}B)^\top. \tag{4}$$

Initial evidence and scope. In a GPU benchmark with $n = 100{,}000$ and a matrix-free KeOps kernel, the extension was faster than standard LOVE in all 200 tested configurations, with a median runtime reduction of approximately 49%. The measured prediction times are shown in Figure 1. Variance agreement was not uniform: in 55 configurations, the relative difference between the two vectors of marginal predictive variances exceeded 10%. Because no exact reference was computed at this size, those differences do not establish which approximation is more accurate. In separate experiments at $n = 10{,}000$ with an exact reference, the extension had lower relative mean error in 46 of 72 cases and lower relative covariance error in 43 of 72 cases. Neither method was uniformly more accurate, and some low-noise cases were difficult for both. The implementation is opt-in and leaves the existing CG and LOVE paths unchanged by default. Its stored-direction path currently supports an unpreconditioned, single-right-hand-side prediction solve. Unsupported cases retain the standard path. The subspace identity above holds in exact arithmetic, whereas finite precision, rank truncation, stopping rules, and different starting vectors can lead to different numerical results.

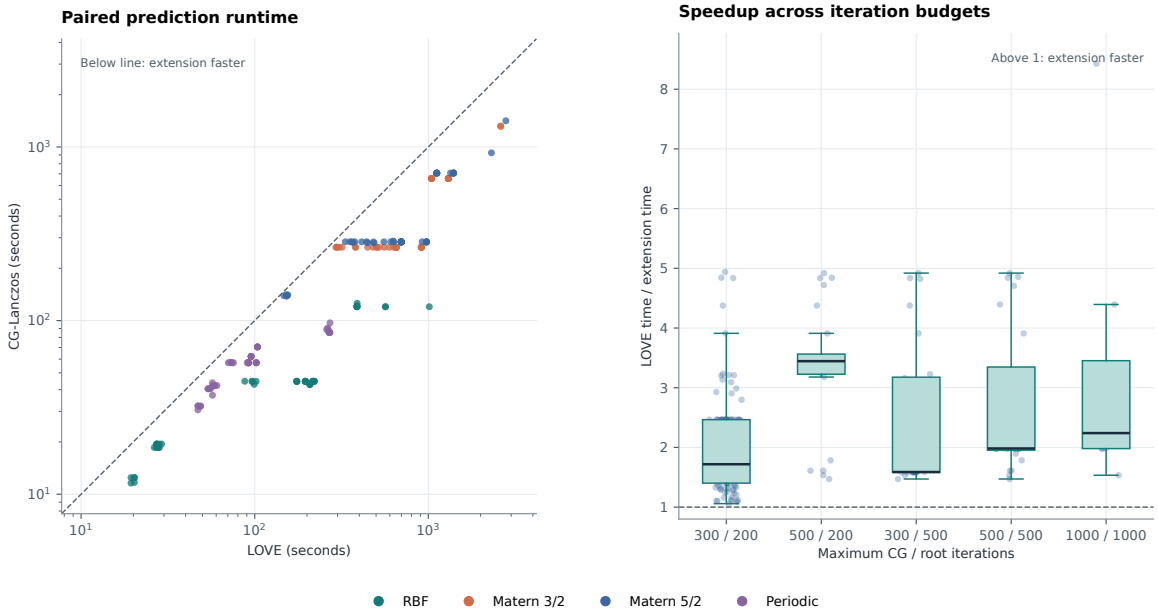


Figure 1: Paired prediction times and speedup across iteration budgets for 200 matrix-free KeOps GPU configurations with $n = 100{,}000$. The extension was faster than LOVE in all 200 configurations, with a median runtime reduction of approximately 49%.

Equivalence of CG and LOVE inverse approximations

Theorem .1 (Equivalence of CG and LOVE inverse approximations)**.** *In exact arithmetic, run unpreconditioned CG on $\widehat{K}w = y$ with $w_0 = 0$, and run Lanczos on $\widehat{K}$ with $q_1 = y/\|y\|_2$. Assume that both algorithms complete J steps without breakdown. If $d_1, \ldots, d_J$ are the CG search directions and (Q_J, T_J) is the Lanczos basis and projected matrix, then*

$$C_J^{\mathrm{CG}} := \sum_{j=1}^{J} \frac{d_j d_j^\top}{d_j^\top \widehat{K} d_j} = Q_J T_J^{-1} Q_J^\top =: C_J^{\mathrm{LOVE}}.$$

Proof. Since we initialize the CG algorithm with $w_0 = 0$, the initial residual and CG search direction are both $d_1 = r_0 = y$. Assuming that the Lanczos algorithm is initialized with $q_1 = y/\|y\|_2$, the first J CG directions and the first J Lanczos vectors therefore span the same Krylov subspace

$$\mathcal{K}_J(\widehat{K}, y) := \mathrm{span}\{y, \widehat{K}y, \ldots, \widehat{K}^{J-1}y\}.$$

Write $D_J = [d_1, \ldots, d_J]$. Because neither algorithm breaks down, there is an invertible $S_J \in \mathbb{R}^{J\times J}$ such that $D_J = Q_J S_J$. Since the CG search directions are $\widehat{K}$-conjugate by construction, i.e., $d_i^\top \widehat{K} d_j = 0$, $i \neq j$, we obtain $D_J^\top \widehat{K} D_J = \mathrm{diag}\left(d_1^\top \widehat{K} d_1, \ldots, d_J^\top \widehat{K} d_J\right)$. Thus,

$$C_J^{\mathrm{CG}} = \sum_{i=1}^{J} \frac{d_i d_i^\top}{d_i^\top \widehat{K} d_i} = D_J (D_J^\top \widehat{K} D_J)^{-1} D_J^\top.$$

Using $D_J = Q_J S_J$ and $T_J = Q_J^\top \widehat{K} Q_J$ then yields

$$C_J^{\mathrm{CG}} = Q_J S_J (S_J^\top Q_J^\top \widehat{K} Q_J S_J)^{-1} S_J^\top Q_J^\top = Q_J T_J^{-1} Q_J^\top.$$

□

References

[1] G. Pleiss, J. R. Gardner, K. Q. Weinberger, and A. G. Wilson, "Constant-Time Predictive Distributions for Gaussian Processes," *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, pp. 4114–4123, 2018.

[2] J. Wenger, G. Pleiss, M. Pförtner, P. Hennig, and J. P. Cunningham, "Posterior and Computational Uncertainty in Gaussian Processes," *Advances in Neural Information Processing Systems*, vol. 35, pp. 10876–10890, 2022.

[3] B. Stankewitz and B. Szabó, "Contraction rates for conjugate gradient and Lanczos approximate posteriors in Gaussian process regression," arXiv:2406.12678, 2024.